

# Individual Report

Name: Kamleshkumar Senthilkumar, Student Number: 24245674

# 1. Reflection and design choices

To begin this project, we started by understanding the domain of **Git** itself. The official Git reference documentation was a very useful starting point. Early on, we mixed up some Git and GitHub concepts. For example, we initially considered including *pull requests* and *forks* in our knowledge graph. After further reading, we realised that these are GitHub features, not part of the core Git model.

## Schema

Most of the domain we needed was already represented in the provided `git_data.py` script. (Repositories, branches, files, commits, and users). After exploring Git documentation, we decided to also include **Tags** in our ontology, as they provide useful versioning information and indicate important milestones within a repository.

Once we had a clear conceptual understanding, we sketched a **UML diagram** to visualise relationships. Initially, we included many inverse properties, but after feedback and testing queries, we removed unnecessary ones and kept only the key relations shown in red in the final diagram.

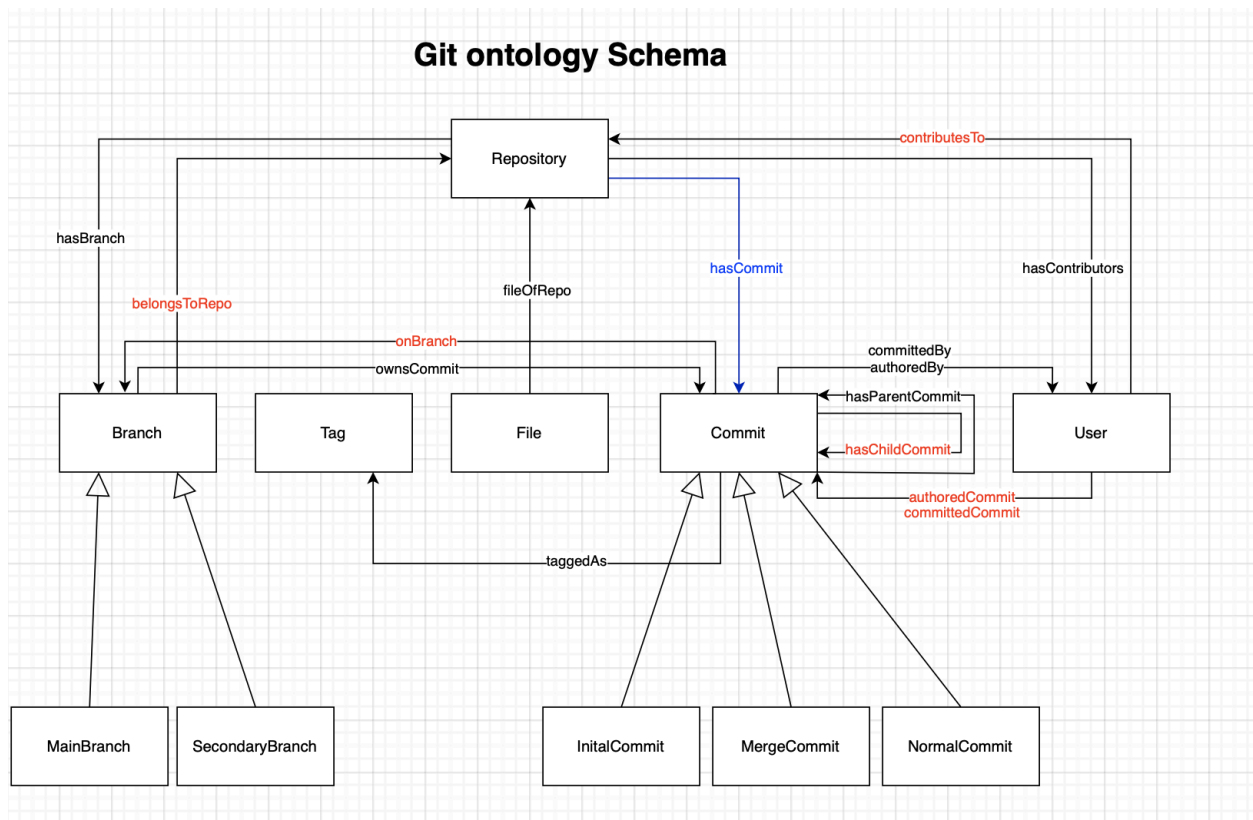


Image 1: an image of the final UML-style representation of our ontology.

### **Subclass and inference handling**

For subclasses, the decisions were straightforward. The MainBranch represents the default branch of a repository (often called “main” or “master”), while SecondaryBranch represents all other branches. The **is\_default** boolean data property was helpful for distinguishing them and was later used directly in SWRL rules. For commits, we used the integer **commit\_parent\_count** and the boolean **is\_initial** to classify them into InitialCommit, NormalCommit, or MergeCommit.

### **Handling Identifier Collisions**

During early population tests, we ran into an issue where many branches shared the same name (“main” or “master”). This caused individuals to merge together when defined as functional data properties. To fix this, we changed the URIs to include functional attributes (which allow for unique identification). For example: **branch\_15580647\_\_master**. This format clearly identifies the branch’s class, repository ID, and name, ensuring uniqueness.

We applied the same approach for commits, using a combination of the repository ID and commit SHA (e.g., **commit\_10123456\_\_bafb6ad**). Although commit SHAs are globally unique in Git, we discovered duplicate entries in our JSON data. Using composite identifiers ensured each commit remained unique across repositories and prevented “inconsistent ontology” errors during reasoning.

## **2. SWRL rules and reasoning**

The SWRL rules used are listed in the user manual. Initially, we tried only using **equivalence relations** alone to classify individuals into subclasses (in Branches and Commit classes). This however, did not fully work at classifying all individuals into the appropriate classes. When we implemented SWRL rules directly in Protégé, we were able to use **SWRLB built-ins** (for numeric comparisons), which were not available through Owlready2. Therefore, we decided to use the **Pellet reasoner** instead of Hermit. Pellet was slower, but performed SWRL rules and reasoning correctly in addition to supporting the built-ins we needed.

Disjoints were included between MainBranch and SecondaryBranch for the Branch class. This was because the Branch could only be of one type. We did not make InitialCommit, NormalCommit, and MergeCommit disjoint due to overlapping classifications. After many-many inconsistent ontology errors and failed queries, we noticed that these individuals can belong to more than one branch. Initial commits can have more than one parent commit if in case an initial commit is made after checking out from an existing branch. This results in a commit having 2 parent branches. In contrast, MergeCommit was restricted to exactly 2 hasParentCommit instead of greater than 2 due to errors in the reasoning (reasoning being strict). After testing and reviewing our data, we noticed that all merge commits in our dataset only had two parents, so we simplified the SWRL rule to use **swrlb:equal(?n, 2)** instead of a greater-than condition.

### 3. Interface Design

For the user interface, we chose to build a **Flask** web application instead of a command-line program. Flask allowed us to present the ontology in a more interactive and accessible way. Users can browse classes, individuals, and query results directly through their browser.

The web interface uses **SPARQL** queries from **RDFlib** and displays both asserted and inferred data. The *Ontology with Python* book was very helpful when building this part. It provided examples for generating routes and search forms that made integration easier.

### 4. Ontology Characteristics and Classes

When deciding on class characteristics, object properties, and datatypes, the *Ontology with Python* book and the *Knowledge Graphs by Hogan et al.* website were both very useful. In the early stages, we added far too many properties and restrictions in an attempt to validate everything strictly. This caused many reasoning errors and inconsistencies, so we later removed unnecessary constraints and kept only the ones that were essential to reasoning and querying.

We also noticed that the reasoning process was heavily affected by data size. With large repositories metadata, the population and reasoning scripts took a very long time to complete. To address this, we selected smaller to medium-sized repositories that still met the dataset requirements but allowed Pellet to finish reasoning within a reasonable time.

### 5. Tools Used

- **Protégé**: Used to design the ontology.
- **Owlready2**: Used in Python to load the ontology, apply reasoning with Pellet.
- **RDFlib**: Used to run SPARQL queries within the Flask application.
- **Flask**: used for web interface for viewing classes and executing queries.
- GitHub REST API: Accessed through the modified `git_data.py` script
- AI was used to help us with the flask application and writing of tedious HTML code. In addition we used it for the python scripts `git_data.py` and `populate_and_reason.py` to debug and also properly ensure loading of data into the ontology.

### 6. Dataset extraction

The dataset extraction was completed using the provided `git_data.py` script, which we extended to include tags. We had to do some research on finding the github token and how to use it as pulling over 20 repos worth of metadata was very time consuming. We picked the Pydevtest organisation and picked up repos from there. This is because there are many of the same contributors working on different projects and repos of the same organisation. Therefore, considering the sizes of the repositories for our project, we went ahead with this choice.

### 3. Time spent on rough tasks and teamworks

#### Time Spent on Rough Tasks and Teamwork

The whole process is difficult to summarise because we didn't complete it on fixed days. There were many changes along the way, and we had to revisit and adjust our design choices quite often. However, designing the knowledge graph and schema took the longest, as building a consistent model with the required relations and properties was difficult to reason about. We had to think through many examples and test different cases.

Extracting the data also took a long time, along with running the reasoning script. These two tasks took about two days, including configuring the script.

Creating the web app was not difficult, as we had the helpful *Ontology with Python* book for reference and had already completed **CITS3403**, so we understood the basics of Flask implementation.

The user manual and individual reports took around a day to complete.

This project was completed in a pair. We chose to work collaboratively on most tasks rather than splitting the work entirely. Both of us contributed to all parts of the project, from ontology design to reasoning and testing.

In general:

- **Kamleshkumar** focused more on ontology design, logical constraints, and SWRL rule testing.
- **Andoni** focused more on developing the Flask application, query execution, and debugging and with documentation

Working on *Git-Onto-Logic* gave me a much deeper understanding of how ontologies can be used to model real-world systems. I think the theory behind it cannot be overlooked — having a strong conceptual understanding made this project much more enjoyable.

I highly recommend external resources such as the *Ontology with Python* book and *Knowledge Graphs* by Hogan et al., as there aren't many tutorials or videos that explain how to design and implement ontologies in a detailed way.

I definitely recommend creating the ontology using **Protégé**, especially for newcomers, as the visualisation really helped to understand the structure and detect issues early.